

✓ Day 20: Batch Normalization in TensorFlow

Key Benefits of Batch Normalization:

1. Stabilizes and speeds up training.
2. Reduces sensitivity to initialization.
3. Acts as a regularizer, reducing overfitting.

1. What is Batch Normalization?

Batch Normalization normalizes the output of each layer by rescaling and shifting it, which leads to faster and more stable training.

✓ 2. Building a Neural Network with Batch Normalization

```
import tensorflow as tf
import numpy as np

# Generating synthetic training data
X_train = np.random.rand(200, 5) # 200 samples, 5 features
y_train = np.random.rand(200, 1) # 200 target values

# Building a neural network with Batch Normalization
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(5,)),
    tf.keras.layers.BatchNormalization(), # Adding Batch Normalization after
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.BatchNormalization(), # Adding Batch Normalization after
    tf.keras.layers.Dense(1) # Output layer
])

# Compiling the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Summary of the model
model.summary()
```



Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_9 (Dense)	(None, 64)	384
batch_normalization_4 (BatchNormalization)	(None, 64)	256

dense_10 (Dense)	(None, 32)	2,080
batch_normalization_5 (BatchNormalization)	(None, 32)	128
dense_11 (Dense)	(None, 1)	33

Total params: 2,881 (11.25 KB)
 Trainable params: 2,689 (10.50 KB)
 Non-trainable params: 192 (768.00 B)

✓ 3. Training the Model with Batch Normalization

```
# Training the model
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2, verbose=1)
```

```
Epoch 1/20
5/5 ----- 2s 51ms/step - loss: 2.7474 - val_loss: 0.3067
Epoch 2/20
5/5 ----- 0s 10ms/step - loss: 1.1077 - val_loss: 0.2292
Epoch 3/20
5/5 ----- 0s 9ms/step - loss: 0.6818 - val_loss: 0.1861
Epoch 4/20
5/5 ----- 0s 10ms/step - loss: 0.4444 - val_loss: 0.1646
Epoch 5/20
5/5 ----- 0s 10ms/step - loss: 0.4354 - val_loss: 0.1568
Epoch 6/20
5/5 ----- 0s 10ms/step - loss: 0.3672 - val_loss: 0.1529
Epoch 7/20
5/5 ----- 0s 11ms/step - loss: 0.2753 - val_loss: 0.1489
Epoch 8/20
5/5 ----- 0s 14ms/step - loss: 0.2264 - val_loss: 0.1407
Epoch 9/20
5/5 ----- 0s 10ms/step - loss: 0.2004 - val_loss: 0.1287
Epoch 10/20
5/5 ----- 0s 9ms/step - loss: 0.1807 - val_loss: 0.1148
Epoch 11/20
5/5 ----- 0s 10ms/step - loss: 0.1689 - val_loss: 0.1027
Epoch 12/20
5/5 ----- 0s 10ms/step - loss: 0.1434 - val_loss: 0.0950
Epoch 13/20
5/5 ----- 0s 10ms/step - loss: 0.1158 - val_loss: 0.0902
Epoch 14/20
5/5 ----- 0s 16ms/step - loss: 0.1203 - val_loss: 0.0861
Epoch 15/20
5/5 ----- 0s 15ms/step - loss: 0.1045 - val_loss: 0.0837
Epoch 16/20
5/5 ----- 0s 11ms/step - loss: 0.1107 - val_loss: 0.0810
Epoch 17/20
5/5 ----- 0s 11ms/step - loss: 0.1248 - val_loss: 0.0790
Epoch 18/20
5/5 ----- 0s 13ms/step - loss: 0.1106 - val_loss: 0.0781
Epoch 19/20
5/5 ----- 0s 9ms/step - loss: 0.1003 - val_loss: 0.0773
Epoch 20/20
5/5 ----- 0s 10ms/step - loss: 0.0937 - val_loss: 0.0771
```

✓ 4. Evaluating the Model Performance

```
# Generating test data
X_test = np.random.rand(50, 5)
y_test = np.random.rand(50, 1)

# Evaluating the model on test data
test_loss = model.evaluate(X_test, y_test)
print(f"Test Loss with Batch Normalization: {test_loss}")
```

```
↗ 2/2 ————— 0s 7ms/step - loss: 0.0839
Test Loss with Batch Normalization: 0.08868569135665894
```

✓ 5. Comparing Training with and without Batch Normalization

```
# Building a model without Batch Normalization for comparison
model_no_bn = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(5,)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)
])

# Compiling and training the model without Batch Normalization
model_no_bn.compile(optimizer='adam', loss='mean_squared_error')
history_no_bn = model_no_bn.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2,
```

```
↗ Epoch 1/20
5/5 ————— 1s 42ms/step - loss: 0.3708 - val_loss: 0.2426
Epoch 2/20
5/5 ————— 0s 14ms/step - loss: 0.3079 - val_loss: 0.1813
Epoch 3/20
5/5 ————— 0s 9ms/step - loss: 0.2260 - val_loss: 0.1350
Epoch 4/20
5/5 ————— 0s 13ms/step - loss: 0.1803 - val_loss: 0.1017
Epoch 5/20
5/5 ————— 0s 9ms/step - loss: 0.1558 - val_loss: 0.0819
Epoch 6/20
5/5 ————— 0s 9ms/step - loss: 0.1262 - val_loss: 0.0758
Epoch 7/20
5/5 ————— 0s 11ms/step - loss: 0.1108 - val_loss: 0.0800
Epoch 8/20
5/5 ————— 0s 12ms/step - loss: 0.1157 - val_loss: 0.0865
Epoch 9/20
5/5 ————— 0s 10ms/step - loss: 0.1179 - val_loss: 0.0905
Epoch 10/20
5/5 ————— 0s 9ms/step - loss: 0.1158 - val_loss: 0.0880
Epoch 11/20
5/5 ————— 0s 9ms/step - loss: 0.1163 - val_loss: 0.0845
Epoch 12/20
5/5 ————— 0s 9ms/step - loss: 0.1066 - val_loss: 0.0820
Epoch 13/20
5/5 ————— 0s 14ms/step - loss: 0.1121 - val_loss: 0.0812
```

```

Epoch 14/20
5/5 ————— 0s 9ms/step - loss: 0.1068 - val_loss: 0.0814
Epoch 15/20
5/5 ————— 0s 10ms/step - loss: 0.1008 - val_loss: 0.0826
Epoch 16/20
5/5 ————— 0s 9ms/step - loss: 0.1065 - val_loss: 0.0842
Epoch 17/20
5/5 ————— 0s 10ms/step - loss: 0.1010 - val_loss: 0.0853
Epoch 18/20
5/5 ————— 0s 9ms/step - loss: 0.1087 - val_loss: 0.0874
Epoch 19/20
5/5 ————— 0s 15ms/step - loss: 0.1081 - val_loss: 0.0892
Epoch 20/20
5/5 ————— 0s 12ms/step - loss: 0.1032 - val_loss: 0.0885

```

✓ 6. Visualizing the Impact of Batch Normalization

```

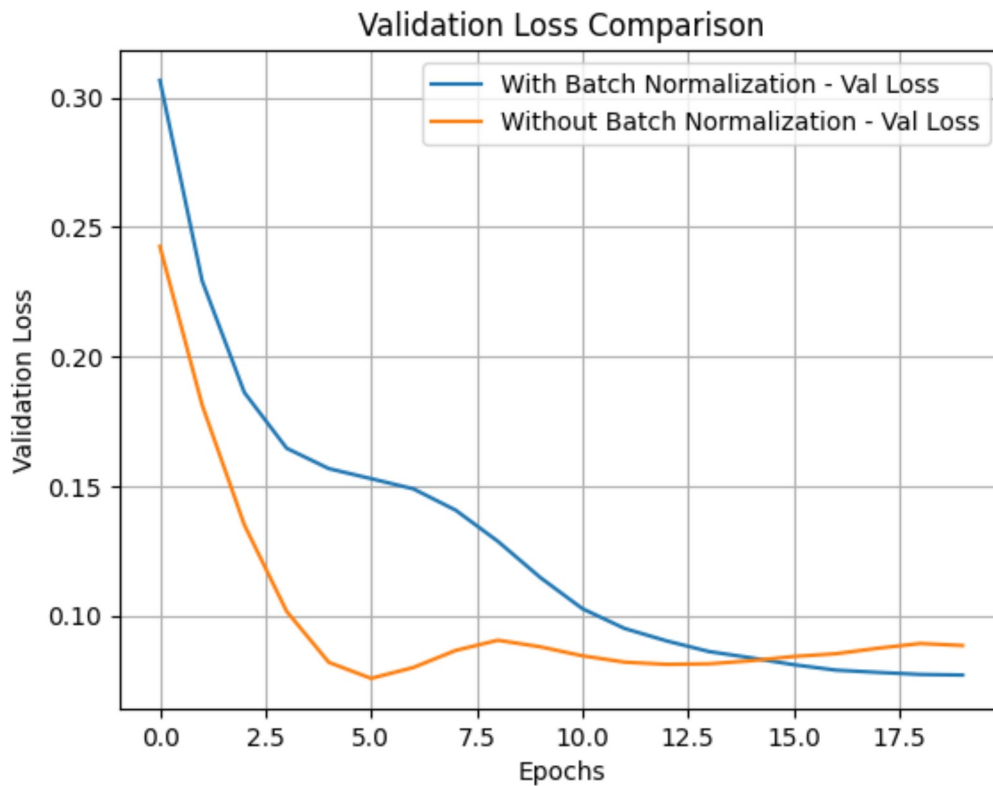
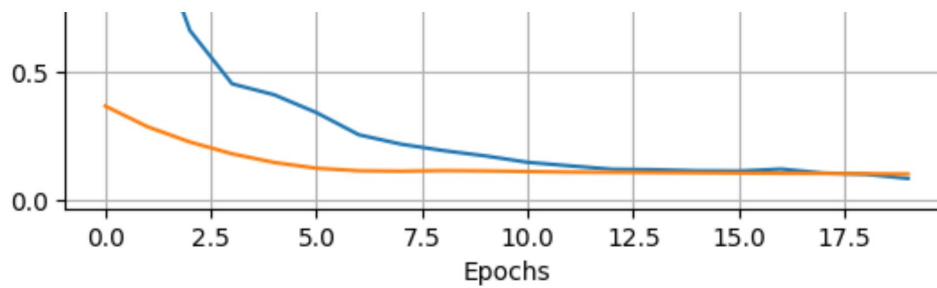
import matplotlib.pyplot as plt

# Plotting the loss curve for both models
plt.plot(history.history['loss'], label='With Batch Normalization')
plt.plot(history_no_bn.history['loss'], label='Without Batch Normalization')
plt.title('Training Loss Comparison')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

# Plotting validation loss
plt.plot(history.history['val_loss'], label='With Batch Normalization - Val Loss')
plt.plot(history_no_bn.history['val_loss'], label='Without Batch Normalization - Val Loss')
plt.title('Validation Loss Comparison')
plt.xlabel('Epochs')
plt.ylabel('Validation Loss')
plt.legend()
plt.grid(True)
plt.show()

```





When and Where to Apply:

1. Use Batch Normalization after dense layers or convolutional layers.
2. It is especially helpful when training deep neural networks.
3. Can sometimes act as a regularizer, reducing the need for Dropout.
4. Keep an eye on the learning rate; Batch Normalization allows you to use higher learning rates for faster convergence.

✓ Conclusion

Batch Normalization significantly stabilizes the training process, reducing overfitting and improving model accuracy. It's a must-have tool when working with deep networks. Whether you're working with large datasets or deep architectures, this technique will enhance your models' performance and make training more efficient.

